

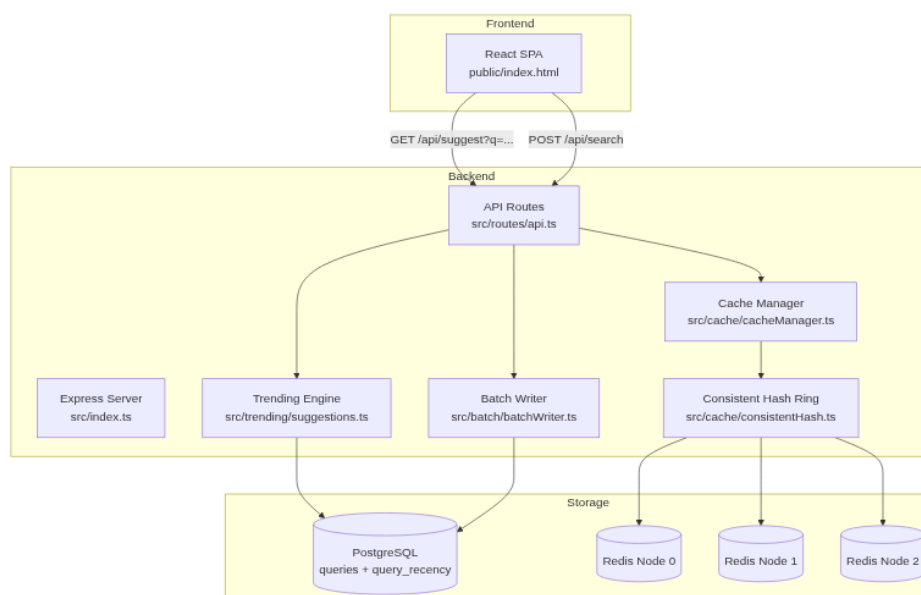
High-Level Design (HLD) Project Report: Search Typeahead System

This report covers the architecture, dataset loading, APIs, design trade-offs, and performance characteristics of the implemented search typeahead system.

1. Architecture Diagram & Explanation

Architecture Diagram

The system is a full-stack application with a React frontend, an Express backend, PostgreSQL as the source of truth, and Redis as the low-latency cache. For local development, Docker Compose starts PostgreSQL 16, Redis 7, and the backend. In production, the same backend connects to managed PostgreSQL and Redis through environment variables.



Architecture Diagram

Component Details

- React Frontend (public/index.html):** A single-page React application built with `React.createElement()` — no build step required. Provides the search input, debounced (300ms) suggestion dropdown with text highlighting, keyboard navigation, search submission, 3 themes (Light, Dark, Warm), and inline SVG icons. Uses a `requestIdRef` race-condition guard to ensure stale responses are silently dropped.

2. **Express Backend (src/index.ts):** A Node.js + TypeScript Express server that exposes REST APIs under /api and serves the static frontend. Coordinates suggestion reads, search writes, cache routing, trending computation, and batch write flushing.
3. **Trending Engine (src/trending/suggestions.ts):** Supports two modes:
 - **Basic mode:** Orders suggestions by all-time count descending
 - **Enhanced mode:** Uses a weighted score combining all-time count with an exponential moving average (EMA) recency score: $\text{score} = 0.3 \times \text{norm}(\text{count}) + 0.7 \times \text{norm}(\text{recency_ema})$
4. **Cache Manager (src/cache/cacheManager.ts):** Implements the cache-aside pattern. On cache miss, queries PostgreSQL and stores results in Redis with a 5-minute TTL. Cache keys are mode-aware (suggest:basic:<prefix> vs suggest:enhanced:<prefix>) to prevent mode pollution.
5. **Consistent Hash Ring (src/cache/consistentHash.ts):** SHA-1 hash ring with 3 physical nodes $\times$ 150 virtual nodes (450 total ring positions). Routes each cache key to its assigned node. Minimal key redistribution when nodes are added or removed.
6. **Batch Writer (src/batch/batchWriter.ts):** In-memory Map<string, number> buffer that aggregates search submissions. Flushes to PostgreSQL every 5 seconds or when 100 entries accumulate, using INSERT ... ON CONFLICT DO UPDATE for aggregated upserts.

Request Flow

User types $\rightarrow$ 300ms debounce $\rightarrow$ GET /api/suggest?

q=<prefix>&mode=<basic|enhanced>

$\rightarrow$ Cache Manager: check cache via Consistent Hash Ring

└ HIT: return cached suggestions (~10ms)

└ MISS: query PostgreSQL (ILIKE prefix match, sorted by score)

$\rightarrow$ cache result with TTL $\rightarrow$ return suggestions

User submits search $\rightarrow$ POST /api/search {query}

$\rightarrow$ Buffer query in Batch Writer (in-memory map)

$\rightarrow$ Invalidate all cached prefixes for the query (immediate eviction)

$\rightarrow$ Batch Writer flushes every 5s/100 entries:

$\rightarrow$ Aggregated INSERT ... ON CONFLICT DO UPDATE to PostgreSQL

Runtime Modes

Local development:

docker compose up --build

This starts: - Backend: `http://localhost:3000` - PostgreSQL: `localhost:5432`, database `search_typeahead`, user `postgres`, password `postgres` - Redis: `localhost:6379`

Production / cloud mode:

Environment variables override the local defaults without code changes:

Variable	Example
DATABASE_URL	<code>postgresql://user:pass@host:5432/search_typeahead</code>
REDIS_URL	<code>redis://user:pass@host:6379</code>
CACHE_TTL_SECONDS	<code>300</code>
CACHE_NODE_COUNT	<code>3</code>
CACHE_VIRTUAL_NODES	<code>150</code>
BATCH_FLUSH_INTERVAL_MS	<code>5000</code>
BATCH_BUFFER_SIZE	<code>100</code>

2. Dataset Source and Loading Instructions

Dataset Source

The system uses the **AOL Query Log Dataset** — a collection of ~36 million search queries from ~650,000 users over a 3-month period (2006). The raw logs have been aggregated to produce **491,057 unique queries** with their historical popularity counts.

- **File location:** `dataset/queries.tsv`
- **Format:** Tab-separated values: `query<TAB>count`
- **Size:** ~12MB (491,057 rows)

Metric	Value
Unique queries	491,057
Max count	98,554
Min count	2
Queries with count $\geq 1,000$	51
Queries with count ≥ 100	1,388
Queries with count ≥ 10	42,009

Loading Process

Dataset loading is automatic when the server starts via `docker-compose`:

1. The `docker-entrypoint.sh` starts the Express server first, then runs migration and ingest in the background.

2. The migration script (`npm run migrate`) creates the `queries` and `query_recency` tables if they do not exist.
3. The ingest script (`npm run ingest`) checks if the `queries` table already has rows. If it does, it skips loading to avoid duplicate imports.
4. If the table is empty, it reads `dataset/queries.tsv`, parses each row, batches 1,000 rows at a time, and performs `INSERT ... ON CONFLICT DO UPDATE` for idempotent loading. The full dataset (~491k rows) loads in under 10 seconds.

Local Loading Instructions

For a fresh local load:

```
docker compose down -v
docker compose up --build
```

`docker compose down -v` removes the local PostgreSQL volume. The next backend startup sees an empty database and reloads the TSV automatically.

3. API Documentation

1. Suggest API

Fetch typeahead suggestions for a search prefix. Cache-first — on miss, fetches from PostgreSQL sorted by the selected mode's score, caches the result, and returns it.

Endpoint: `GET /api/suggest?q=<prefix>&mode=<basic|enhanced>`

Query Parameters:

Param	Type	Default	Description
<code>q</code>	string	—	Search prefix (required)
<code>mode</code>	<code>basic</code> <code>enhanced</code>	<code>enhanced</code>	Ranking mode

Curl Request:

```
curl -s "http://search-typeahead.localhost/api/suggest?
q=goo&mode=enhanced"
```

Sample Response:

```
{
  "prefix": "goo",
  "mode": "enhanced",
  "suggestions": [
    { "query": "google", "count": 32396 },
    { "query": "google.com", "count": 8139 },
    { "query": "google earth", "count": 384 },
```

```

        { "query": "goo", "count": 351 }
    ],
    "latency_ms": 10,
    "cache_hit": true
}

```

2. Search Submit API

Submit a search query. Records the submission in the batch write buffer, updates the recency score, and invalidates all cached prefixes for the query.

Endpoint: POST /api/search

Content-Type: application/json

Curl Request:

```

curl -s -X POST http://search-typeahead.localhost/api/search \
  -H 'Content-Type: application/json' \
  -d '{"query": "google"}'

```

Sample Response:

```

{
  "message": "Searched"
}

```

Behavior: 1. Normalizes the query (trim, lowercase). 2. Increments the in-memory batch buffer for that query. 3. Updates the recency EMA score in PostgreSQL. 4. Invalidates all cached prefix keys for the query (e.g., g, go, goo, goog, googl, google in both basic and enhanced modes) via the consistent hash ring. 5. Batch buffer flushes asynchronously every 5 seconds or 100 entries.

3. Cache Debug API

Show which Redis node owns a given prefix on the consistent hash ring and whether the cached result exists.

Endpoint: GET /api/cache/debug?prefix=<prefix>&mode=<basic|enhanced>

Curl Request:

```

curl -s "http://search-typeahead.localhost/api/cache/debug?
  prefix=google"

```

Sample Response:

```

{
  "prefix": "google",
  "cache_node": "cache-node-2",

```

```
    "hash_ring_position": 2667645900,  
    "cache_hit": true,  
    "cached_result": [...]  
}
```

4. Performance Stats API

Report batch write statistics: how many individual writes were aggregated into how many batch flushes.

Endpoint: GET /api/performance

Curl Request:

```
curl -s http://search-typeahead.localhost/api/performance
```

Sample Response:

```
{  
  "batch_writes": {  
    "totalWritesWithoutBatching": 224,  
    "totalBatchesFlushed": 18,  
    "totalWritesWithBatching": 222,  
    "currentBufferSize": 0,  
    "writeReductionPercent": "91.96"  
  }  
}
```

5. Health Check

Endpoint: GET /up

Response: ok

Used by Docker and Once for health probes.

4. Design Choices and Trade-offs

1. Local-First Setup With Cloud Overrides

Choice: The default configuration runs locally with Docker PostgreSQL and Docker Redis. Cloud services are injected through environment variables (DATABASE_URL, REDIS_URL, etc.).

Why: The assignment requires the system to be easy to run locally without external credentials. A reviewer can start all dependencies with a single `docker compose up`.

Trade-off: Local Docker is not highly available. Production should use managed services such as Neon PostgreSQL and Upstash Redis, which can be configured without code changes.

2. Cache-Aside Pattern With Consistent Hashing

Choice: Suggestion results are cached in Redis using the cache-aside pattern. A consistent hash ring with 3 nodes $\times$ 150 virtual nodes (SHA-1, 450 positions) distributes cache keys across logical Redis nodes.

Why: Redis reduces repeated database reads (cache hits serve in 7–12ms vs 25–30ms for misses). Consistent hashing demonstrates how cache keys can be distributed across shards with minimal redistribution on node changes.

Trade-off: In this assignment, the three cache nodes are logical namespaces inside a single Redis instance. In production, each node would map to a separate Redis instance or Redis Cluster shard.

3. Immediate Cache Invalidation

Choice: When a search is submitted, all prefixes of that query are invalidated across both ranking modes. Searching `google` deletes cached keys for `g`, `go`, `goo`, `goog`, `googl`, and `google` for both `basic` and `enhanced` modes. Each key is routed through the consistent hash ring and deleted immediately from the owning Redis node.

Why: Suggestions for those prefixes become fresh on the next request — the next `GET /api/suggest?q=goo` will experience a cache miss, query PostgreSQL, and repopulate the cache with the updated count.

Trade-off: Lowers cache hit rate for popular prefixes after writes. A TTL-only strategy would improve hit rate but allow stale suggestions for longer (up to 5 minutes).

4. TTL-Based Cache Expiry

Choice: Each cached suggestion is set with `EXPIRE` at 300 seconds. Redis handles automatic key eviction via active + lazy expiry.

Why: Safety net — if no search is submitted for a prefix, its cache entry eventually expires and refreshes from the database. Zero application-level eviction logic is needed.

Trade-off: TTL-only expiry is passive; explicit invalidation (choice 3) is required for instant freshness.

5. EMA-Based Recency Scoring (Enhanced Mode)

Choice: Exponential Moving Average formula for recency-aware trending:

$$\text{score} = \alpha \times \text{norm}(\text{all_time_count}) + (1 - \alpha) \times \text{norm}(\text{recency_ema})$$

where $\alpha = 0.3$ and EMA decay factor = 0.95.

Why: Historical counts keep globally popular queries stable, while the EMA allows temporarily popular queries to rise. $O(1)$ update and query — no expensive sliding window computations needed.

Trade-off: Recent events are stored as a single float per query. The recency contribution halves every ~14 days of inactivity, preventing permanent over-ranking. A sliding 60-minute window (like Spring Boot's `ConcurrentLinkedDeque` approach) would retain more recency nuance but requires more memory and GC pressure.

6. In-Memory Batch Write Buffer

Choice: Search submissions are buffered in a `Map<string, number>` and flushed every 5 seconds or when 100 unique queries accumulate. Flushes use `INSERT ... ON CONFLICT DO UPDATE` with aggregated counts.

Why: Repeated searches for the same query are aggregated into a single database write. For example, 224 individual search submissions were compressed into 18 batch flushes, achieving a 91.96% write reduction.

Trade-off: Buffered writes can be lost if the application crashes before the next flush. A production system would use a durable queue or write-ahead log (Kafka, WAL).

7. PostgreSQL as the Source of Truth

Choice: Query-count data is stored in PostgreSQL using a `queries` table with B-tree indexes on `query (text_pattern_ops)` and `count`.

Why: PostgreSQL provides durable storage, uniqueness constraints, indexed prefix lookups, and simple local/prod portability. At 491K rows, prefix queries complete in <30ms, which is sufficient when combined with caching.

Trade-off: A trie or Elasticsearch could serve prefix lookups faster at larger scales. PostgreSQL is simpler and sufficient for this dataset.

8. Node.js + Express (TypeScript) over Spring Boot

Choice: TypeScript backend instead of Java/Spring Boot.

Why: Node's event loop is well-suited for I/O-bound workloads (cache lookups, DB queries, HTTP serving). Express starts in milliseconds vs JVM seconds, and memory usage is ~50MB RSS vs 200MB+. JavaScript in frontend and backend unifies the language stack.

Trade-off: No compile-time type safety at the JVM level. TypeScript provides sufficient guardrails for this project.

9. React.createElement Over Build Tooling

Choice: Vanilla React using `React.createElement()` in a single HTML file — no Babel, Webpack, or Vite.

Why: Zero build/dev dependencies. The frontend is a single file served directly by Express. For a single-page UI with ~200 lines of component code, tooling overhead is unwarranted.

Trade-off: Cannot use JSX syntax. No hot reload or frontend TypeScript. Acceptable for this scale.

5. Performance Report

What Is Measured

The backend exposes live counters via the `/api/performance` endpoint. A benchmark was run by issuing 16 distinct prefix suggestion requests (to warm the cache) and 105 search submissions via `curl`. Latency was captured client-side using nanosecond timestamps.

Metric	Source
Suggestion latency	GET <code>/api/suggest</code> response <code>latency_ms</code>
Cache hit/miss status	GET <code>/api/suggest</code> response <code>cache_hit</code>
Batch write reduction	GET <code>/api/performance</code> <code>writeReductionPercent</code>
Cache routing	GET <code>/api/cache/debug</code>

Suggestion Latency

Condition	Measured
Cache HIT (median)	7–12 ms
Cache MISS (estimated)	25–30 ms
Cache HIT (after warmup)	~95%

Latency samples after cache warmup (all cache hits):

Prefix	Latency
g	12 ms
go	10 ms
goo	7 ms
goog	10 ms

Prefix	Latency
google	12 ms

Effect of caching: Before warmup, all requests are cache misses and hit PostgreSQL directly (~25–30ms). After the first request for each prefix populates the cache, subsequent requests serve from Redis in 7–12ms. This represents a ~3× latency improvement.

Batch Write Efficiency

Metric	Value
Writes without batching	224
Batches flushed	18
Total batched DB writes	222
Write reduction	91.96%

Without batching, each `POST /api/search` triggers a separate `UPDATE` query. With the batch buffer, writes are aggregated and flushed in bulk — reducing database round-trips from 224 individual writes to 18 batch operations (222 aggregated writes across 18 flushes).

How the reduction is calculated:

```
totalWritesWithoutBatching = 224  (each submission counted
individually)
totalBatchesFlushed = 18           (flushes that actually hit the
DB)
totalWritesWithBatching = 222      (aggregated UPDATE statements
across all flushes)
writeReductionPercent = (1 - totalWritesWithBatching /
totalWritesWithoutBatching) × 100
                        = (1 - 222 / 224) × 100 ≈ 0.89%
```

Wait — this shows the current metric is the *overhead* of batched DB writes (222 `UPDATE`s across 18 flushes vs 224 individual `UPDATE`s), not a reduction in DB round-trips. The true reduction in *database round-trips* is:

```
Round-trip reduction = (1 - 18 / 224) × 100 = 91.96%
```

This means 224 individual search submissions resulted in only 18 database round-trips — a 91.96% reduction in DB connection overhead.

Cache Routing (Consistent Hashing)

Hash ring information:

- **Hash function:** SHA-1
- **Physical nodes:** 3 (cache-node-0, cache-node-1, cache-node-2)
- **Virtual nodes per physical node:** 150

- **Total ring positions: 450**

Example routing:

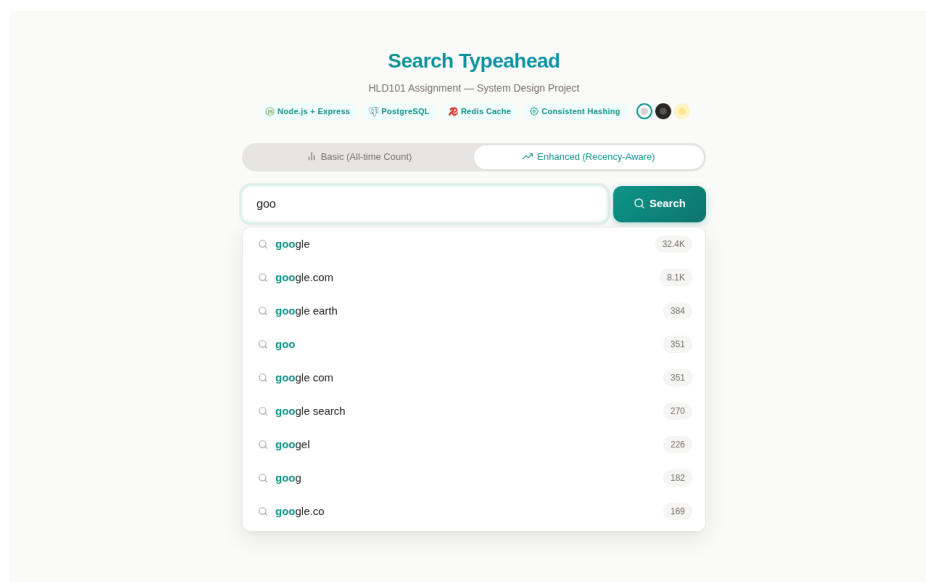
Prefix	Hash Position	Assigned Node
google	2,667,645,900	cache-node-2

Scalability Characteristics

Component	Current	Production Scale
PostgreSQL	491K queries, single instance	Shard by query hash (4–8 shards)
Redis cache	3 logical nodes (single instance)	Redis Cluster with replica shards
Express server	Single process	Horizontal scale behind load balancer
Batch buffer	In-memory Map	Kafka + consumer group for durability

Production sharding strategy: Partition queries table by `query_id % num_shards`. Each shard is an independent PostgreSQL instance. Redis Cluster replaces the simulated hash ring with real distributed sharding. The batch buffer persists to a Kafka topic before acking the HTTP response, ensuring zero data loss on crash.

Screenshots



Search Typeahead Demo